From Electrons to Apps

Levels of Abstraction in Computing

A 1-hour journey from physics to the apps you use every day



🚀 Welcome!

Ever wonder what *actually* happens when you double-click an icon?

We'll zoom in all the way to the electrons flowing through a CPU's transistors — then zoom back out, floor by floor, through bits, logic gates, machine code, C, modern languages, operating systems, apps — and even "the cloud" (spoiler: it's just someone else's computer).

No prior programming knowledge needed — just curiosity.
By the end, every future "how does this work?" question gets a little less mysterious.

What actually happens when you double-click an icon?

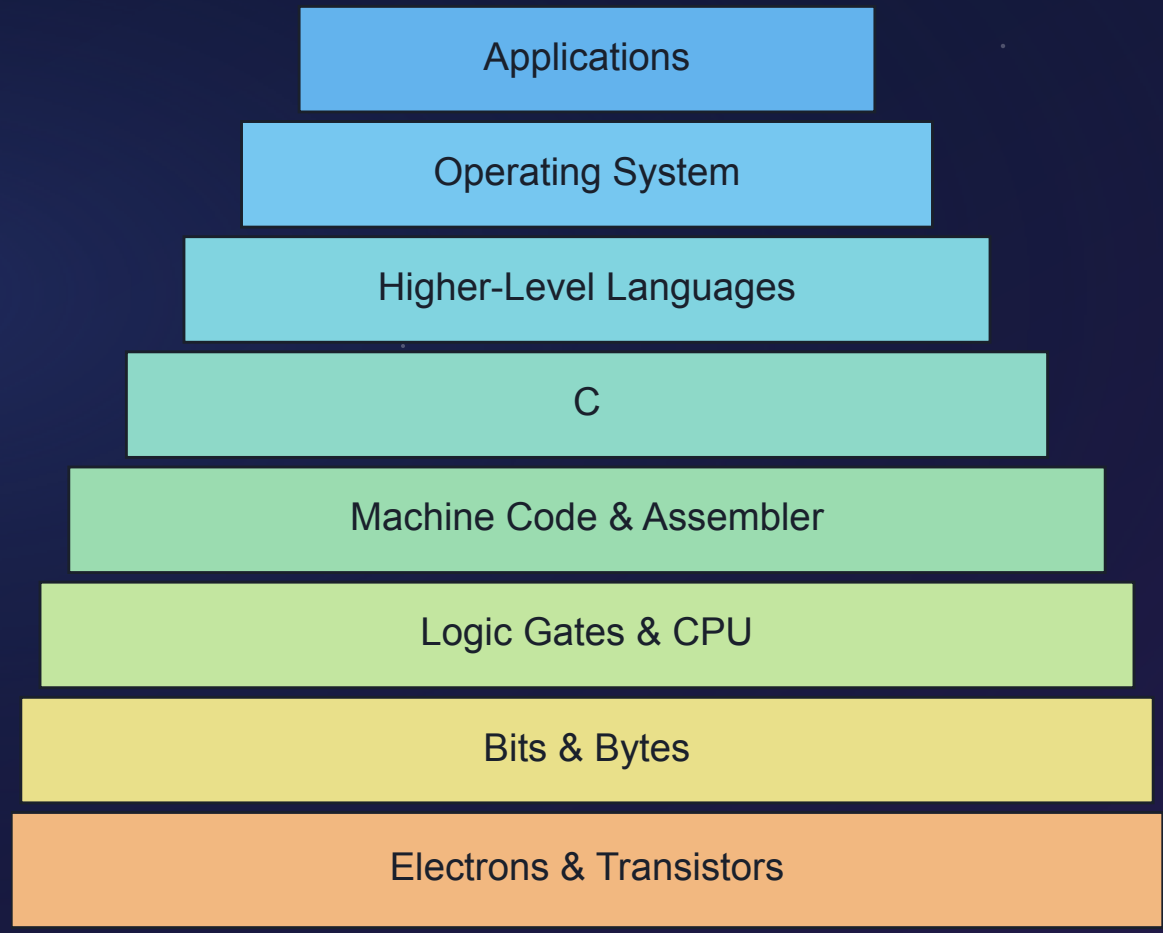
- Somewhere, **electrons** start moving in a very specific pattern
- By the end of this hour, you'll be able to explain *every step* between "electron" and "app opens"

The Abstraction Tower 🗼

We'll climb it floor by floor:

1. Electrons & Transistors
2. Bits & Bytes
3. Logic Gates → CPU
4. Machine Code & Assembler
5. C — the "portable assembler"
6. Higher-Level Languages
7. Operating Systems
8. User Applications
9. Bonus: The Cloud

(This slide comes back between every section — watch the current floor light up)



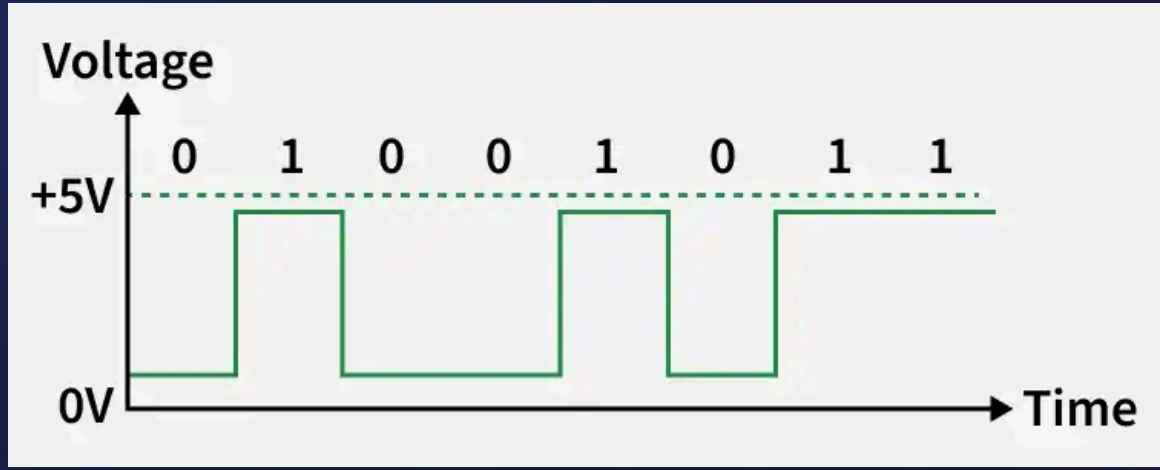
1. Electrons & Transistors

Floor 0 — Physics

It starts with electricity

- Electricity = electrons flowing through a wire
- We can measure it as **voltage**: high voltage or low voltage
- Computers only care about **two states**: voltage present or not
  - High voltage → **1**
  - Low voltage → **0**

A stream of 1s and 0s is really just voltage switching high and low over time



The transistor: a switch controlled by electricity

- A transistor is a tiny switch
- Normally, switches are flipped by hand (light switch)
- A transistor is flipped by **another electrical signal**
- This means: *electricity can control electricity*

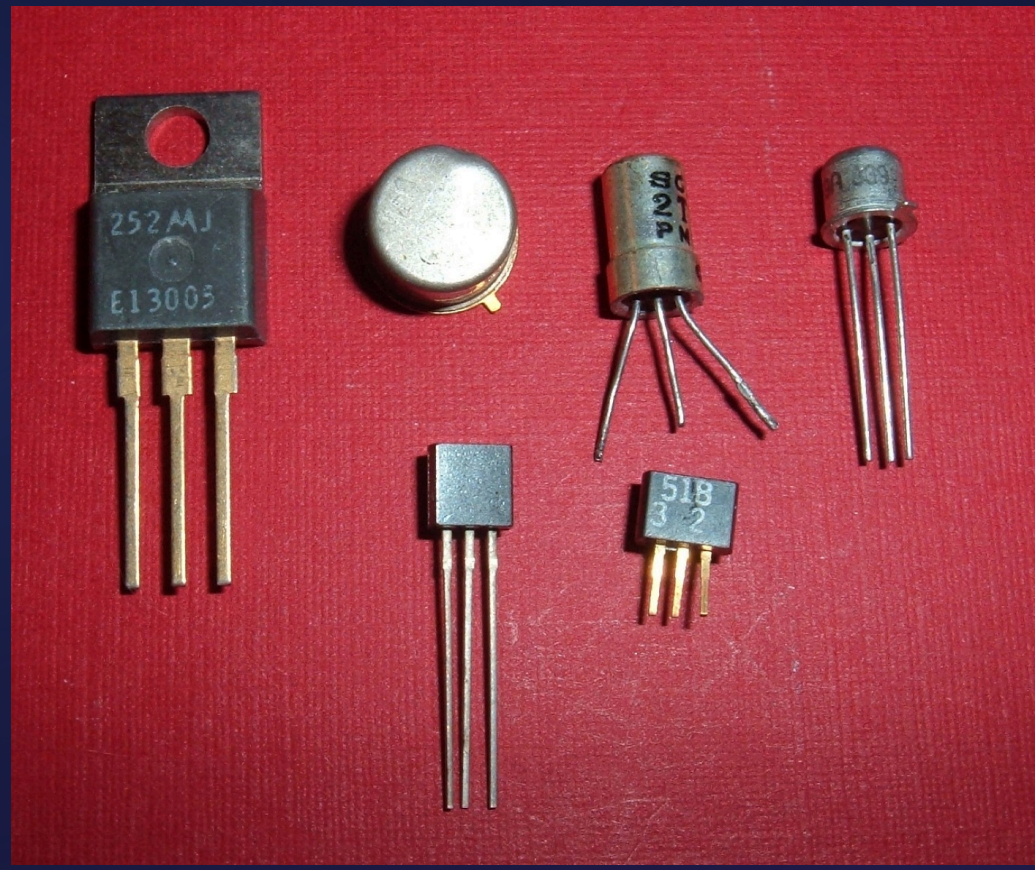


Photo: real discrete transistors — modern chips shrink these to billions per chip

Why does that matter?

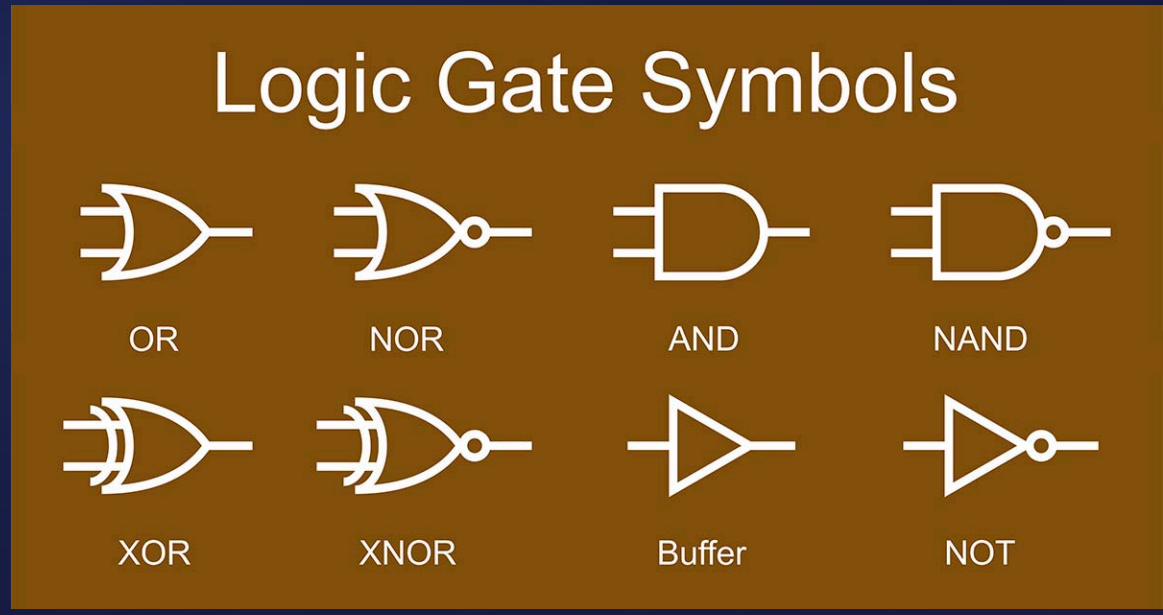
- If one signal can turn another on/off...
- ...we can chain millions of them together
- Each transistor makes a tiny **decision**: pass current, or don't

From switches to logic gates

- Combine a few transistors → build a **logic gate**
- Logic gates: AND, OR, NOT
  - AND → true only if *both* inputs are on
  - OR → true if *either* input is on
  - NOT → flips the signal

Key idea: everything below this point is just physics turning into a yes/no signal.

Video: a 1-minute visual explainer of logic gates



🔗 Bridge: Floor 0 → Floor 1

From switches to information

- **We just built:** transistors combined into logic gates (AND/OR/NOT) that turn electricity into a reliable yes/no signal
- **Next question:** what do we actually *do* with billions of these signals?
- **Answer:** we group them into **bits** and **bytes** to represent real information — numbers, letters, colors, sound

A glass of water is neither half full nor half empty — it just is as it is.
It's humans who give it meaning. The same wire is just carrying electricity —
it's humans who agree to call high voltage "1" and low voltage "0".

2. Bits & Bytes

Floor 1 — Representing Information

The bit

- **Bit** = one on/off signal = one 0 or one 1
- The smallest unit of information a computer can store



The byte

- **Byte** = a group of 8 bits
- Example: `0000 0001`
- A single byte can represent 256 different values (0–255)

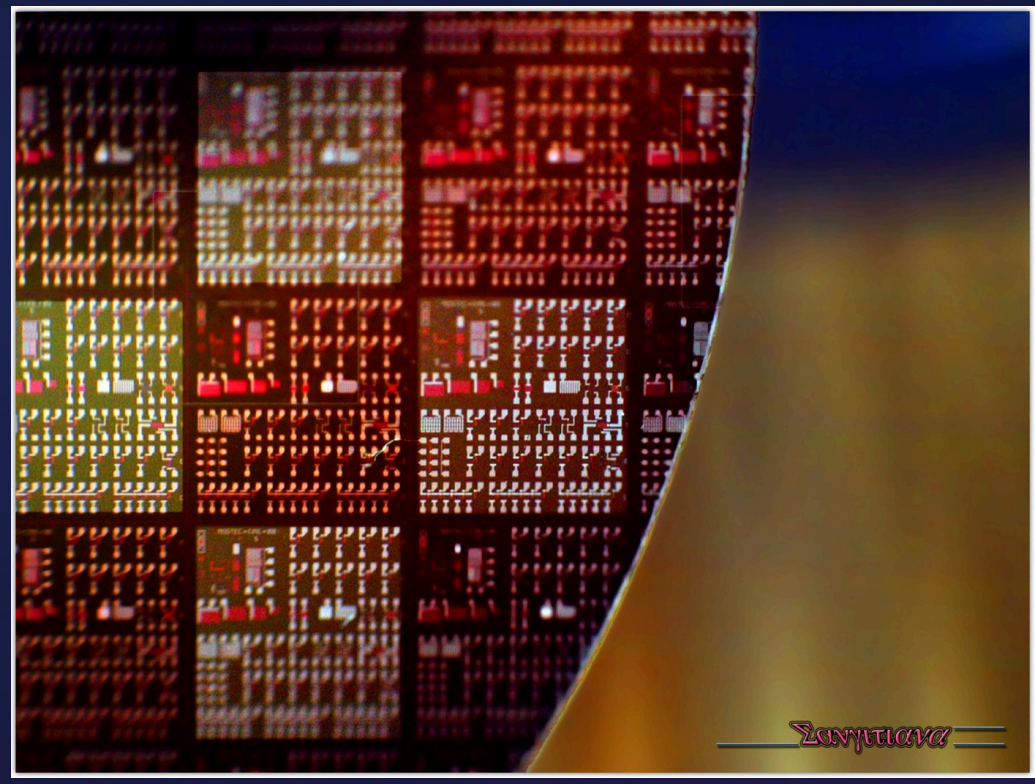
```
0000 0000  = 0
0000 0001  = 1
0000 0010  = 2
0000 0011  = 3
...
1111 1111  = 255
```

Why binary, not decimal?

- Building a reliable switch with exactly 2 states (on/off) is easy
- Building a reliable switch with 10 states would be very hard
- Binary isn't a choice for convenience — it's a choice for **reliability**

😄 *There are 10 types of people in the world: those who understand binary, and those who don't.*

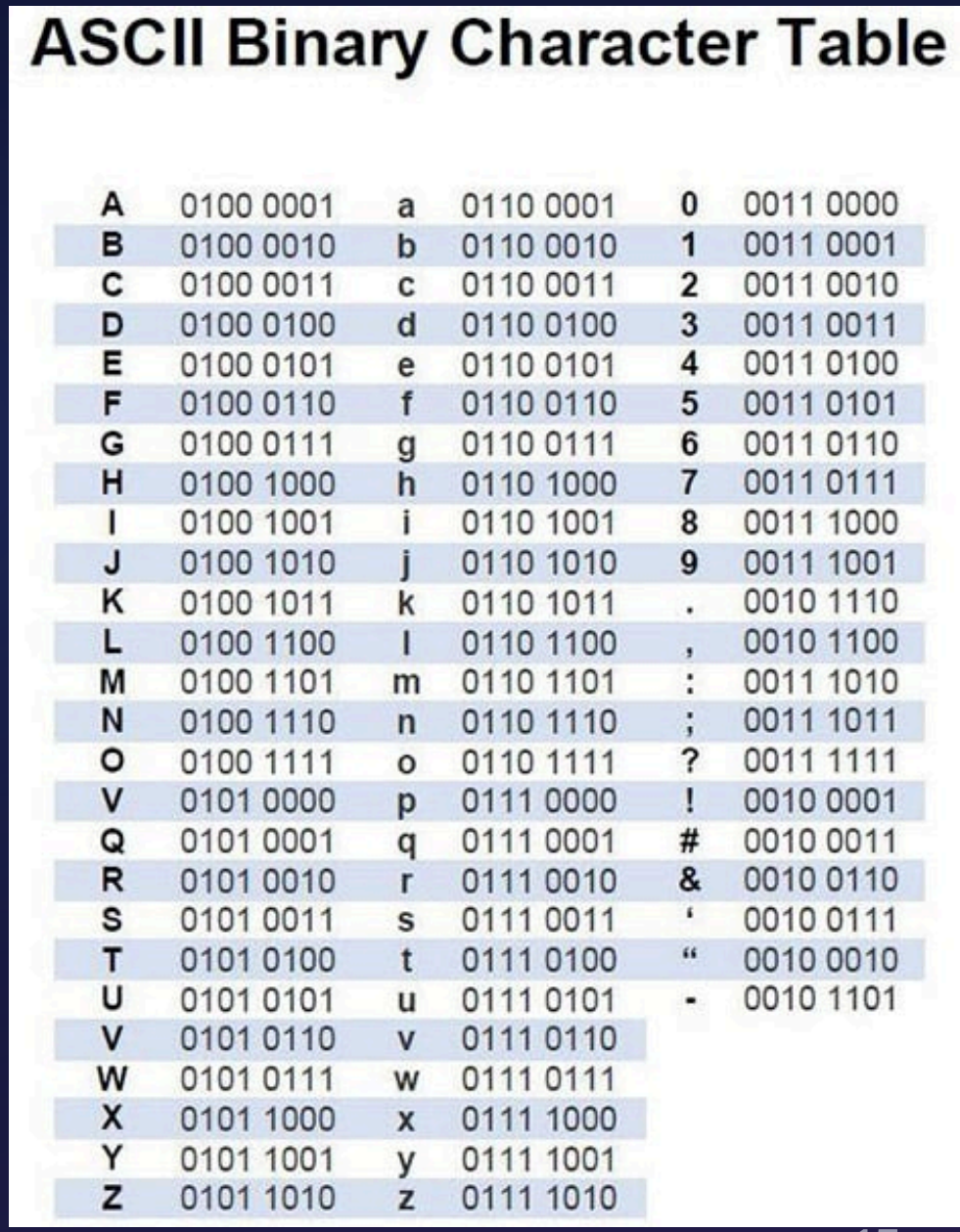
Photo: a silicon wafer — the raw material chips are cut from



Bits represent *everything*

- Numbers → straightforward binary counting
- Letters → each letter mapped to a number (e.g. ASCII: 'A' = 65)
- Colors → red/green/blue values as numbers
- Sound → sampled amplitude values, thousands of times per second

Key idea: bits don't "know" what they mean — the program interpreting them decides.



🔗 Bridge: Floor 1 → Floor 2

From storing information to computing with it

- **We just learned:** bits and bytes let us *represent* numbers, letters, colors, and sound
- **Next question:** representing data is nice, but how do we *calculate* with it — add two numbers, compare them, move them around?
- **Answer:** wire logic gates together into circuits that actually compute — that's how a **CPU** is built

3. Logic Gates → CPU

Floor 2 — Building a Calculator from Switches

Combining gates: an adder

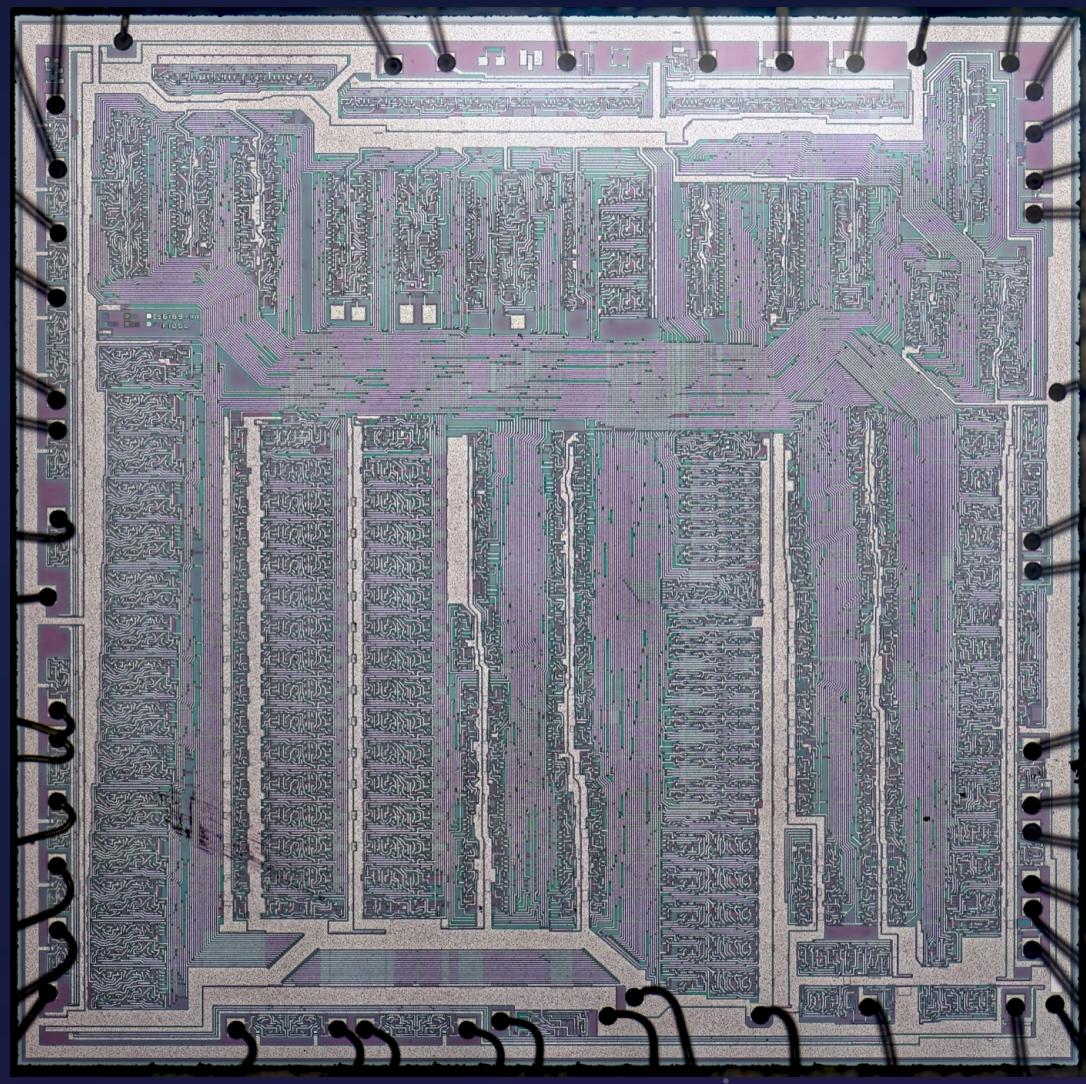
- Take two 1-bit numbers, want to add them
- A small circuit of AND/OR/NOT gates can compute the sum + carry
- This is a real circuit: **the binary adder**

Zoom out: millions of these

- Chain adders together → add whole bytes, whole numbers
- Add more circuits → multiply, compare, move data around
- Package **millions to billions** of transistors into one chip

That chip is the CPU

Photo: a real microprocessor die, magnified — every rectangle is functional circuitry



Two more ingredients

- **Registers** — tiny, extremely fast storage slots inside the CPU
- **Clock** — a ticking signal that paces every step
  (modern CPUs tick billions of times per second — GHz)

Key idea: the CPU is a huge, fast calculator made entirely of on/off switches.

🔗 Bridge: Floor 2 → Floor 3

From a calculator to a set of commands

- **We just built:** a CPU — millions of gates, registers, and a clock, all able to do arithmetic on bits
- **Next question:** a calculator alone does nothing — how do we *tell* it what to calculate, and in what order?
- **Answer:** we give it a sequence of **instructions** it understands — machine code, written more conveniently via assembler

4. Machine Code & Assembler

Floor 3 — Talking to the CPU

Machine code

- The CPU only understands raw binary instructions
- Example (simplified): `10110000 01100001`
  → "put the value 97 into register A"
- Every CPU family has its own machine code "dialect"

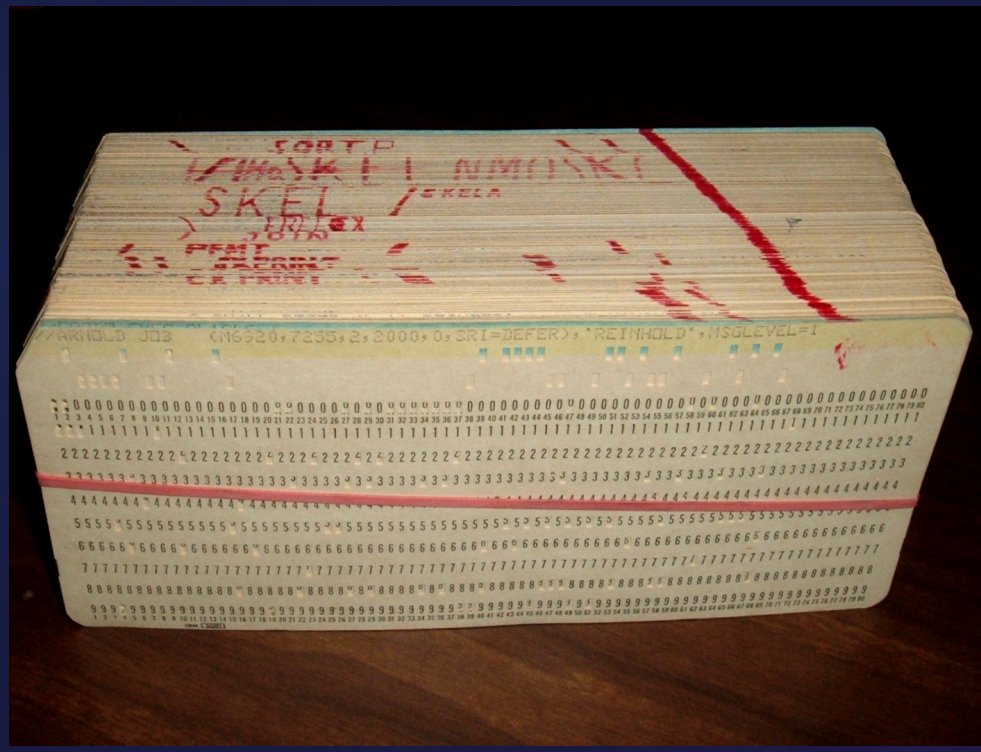


Photo: a punched card deck — an early, physical way of feeding instructions to a computer

⚠️ Data and instructions look identical

- Memory just stores **bytes** — `01100001` could mean:
  - the number 97
  - the letter `a`
  - or the instruction "add register B to register A"
- The CPU doesn't "know" which one it is — *context* decides
- Programs and data live in the **same memory**, side by side

If a program mistakes data for instructions (or vice versa), the CPU happily executes whatever garbage bytes it finds — often ending in a crash or an infinite loop, i.e. the computer **freezes**

This mix-up is also the root cause of classic security bugs like buffer overflows

Assembler: a thin translation layer

- Machine code is unreadable for humans
- **Assembly language** gives instructions short names:

```
MOV A, 97   ; put 97 into register A
ADD A, B    ; add register B to register A
JMP loop    ; jump back to "loop"
```

- Each assembly line maps almost 1:1 to one machine code instruction

Key idea: assembler is a direct, human-readable translation of machine code.

🔗 Bridge: Floor 3 → Floor 4

From one CPU's dialect to a portable language

- **We just saw:** assembler instructions map almost 1:1 to machine code — but only for *one specific* CPU type, and they're tedious to write
- **Next question:** what if we want to write code once and run it on many different CPUs, with less tedium and fewer mistakes?
- **Answer:** introduce a language with structure — variables, functions, loops — that a **compiler** translates to whichever CPU's machine code we need

5. C — the "Portable Assembler"

Floor 4 — Structured, Reusable Code

The problem with assembly

- Tied to *one specific* CPU type
- Extremely tedious to write
- Very easy to make mistakes (no safety nets)

😄 *Assembly programmers are the people who hike without GPS or trail markers and insist it builds character. C came along and said: maybe a trail map wouldn't hurt.*

Enter C

```c
int add(int a, int b) {
    return a + b;
}
```

- Introduces **variables**, **functions**, **loops** as readable text
- Still maps closely to what the machine actually does
- Nicknamed "portable assembler" — same ideas, works on many CPUs

The compiler

- A program that translates C source code → machine code
- Different compiler per target CPU, same C source code
- This is what makes C "portable"

Source code vs. the executable

- **Source code** (`add.c`) is a text file — human-readable, editable, means nothing to the CPU on its own
- The **compiler** reads it and produces a **binary executable** (`add.exe`, `add`, or similar) — pure machine code, ready for the CPU
- They are **completely different files**:

	Source code	Executable
Format	plain text	binary (machine code)
Readable by	humans (and editors)	the CPU
Can you run / edit it?	no / yes	yes / not really

- Edit the `.c` file → you must **recompile** to get an updated binary

😄 *Forgetting to recompile is like changing the recipe but serving yesterday's cake.*

Where C lives today

- Operating system kernels (Linux, parts of Windows/macOS)
- Embedded systems (microwaves, cars, routers)
- Performance-critical software



🔗 Bridge: Floor 4 → Floor 5

From close-to-the-machine to close-to-human-thinking

- **We just saw:** C is portable and structured, but you still manage memory yourself, and it takes a lot of code to express simple ideas
- **Next question:** can we write code that reads more like plain thinking, and let the computer handle the tedious bookkeeping?
- **Answer:** higher-level languages — Python, JavaScript, Java, and many more — trade a little control for a lot of readability and speed

6. Higher-Level Languages

Floor 5 — Closer to Human Thinking

The problem C still has

- You manage memory yourself (easy to get wrong)
- Verbose — lots of code for simple ideas
- Still thinking like the machine, not like a human

Higher-level languages

- Python, JavaScript, Java, and many others
- Automatic memory management — the language cleans up after you
- Rich ecosystems: huge libraries for almost any task
- Example — same idea as our C function, in Python:

```
def add(a, b):
    return a + b
```

😄 *In C, managing memory yourself is like being responsible for cleaning up every room you ever walk into, forever. Higher-level languages hired you a cleaning service — it's called garbage collection.*

Compiled vs. interpreted (just the vocabulary)

- Some languages are **compiled ahead of time** (like C)
- Some languages are **interpreted** — translated and run on the fly
- Many modern languages do a mix of both
- *(We'll go deeper into this in a future lecture)*

😄 *Compiled languages translate the whole book before you start reading. Interpreted languages translate it sentence by sentence while you read — slower, but great if you keep changing your mind about the plot.*

Key idea: every layer trades a bit of performance/control for speed of writing and readability.

🔗 Bridge: Floor 5 → Floor 6

From one program to many programs sharing one machine

- **We just learned:** higher-level languages let us write programs quickly, in whichever language fits the job
- **Next question:** your computer runs *many* programs at once (browser, music, this slideshow) — who decides which one gets the CPU, memory, and screen right now?
- **Answer:** the **operating system** — it manages the hardware and shares it safely between every running program

7. Operating Systems

Floor 6 — Sharing the Machine

What does an OS actually do?

- Manages the hardware: CPU, memory, disk, screen, network
- Shares that hardware safely between **many** running programs
- Without it, every app would need to know how to talk to every device directly

A few key concepts

- **Process** — a running program, with its own slice of memory and CPU time
- **File** — a named chunk of data stored on disk, managed by the OS
- **Driver** — small piece of software that lets the OS talk to specific hardware

Same idea, different implementations

- **Windows**, **macOS**, **Linux** (and Android, which is built on Linux)
- All solve the same core problems: process management, files, drivers, security
- Different design choices, same fundamental job



Tux the penguin — mascot of the Linux kernel

📽️ Demo: seeing processes live

- Open Task Manager (Windows) or Activity Monitor (macOS)
- Point out: dozens of processes running *right now*
- This is the OS doing its job, visibly, in real time

🔗 Bridge: Floor 6 → Floor 7

From managing hardware to what you actually see

- **We just learned:** the OS manages processes, files, and drivers, and shares the hardware between programs
- **Next question:** the OS runs programs, but what *are* those programs, from your point of view as a user?
- **Answer:** they're the **applications** — the browser, the game, the word processor — everything you actually click on and use

8. User Applications

Floor 7 — What You Actually See

Apps are built on everything below

- A browser, a game, a word processor:
  - Written in a higher-level language (or C/C++)
  - Compiled/interpreted down through the layers
  - Uses OS services (files, network, screen, input)
  - Ultimately runs as CPU instructions
  - Which are, in the end, electrons flipping switches — billions of times per second



Full circle 🔄

Double-click icon

→ OS loads the program into memory

→ CPU starts executing its instructions

→ Instructions are just patterns of bits

→ Bits are just voltage levels

→ Voltage levels are just electrons moving through transistors

🔗 Bridge: Floor 7 → Bonus Floor

From your device to every device

- **We just closed the loop:** on *your* device, apps run all the way down to electrons — a complete, self-contained tower
- **Next question:** but many apps you use daily (weather, chat, video) clearly involve *other* computers too — where are those, and how do they fit in?
- **Answer:** those other computers live in data centers — what everyone calls **"the cloud"**

9. The Cloud

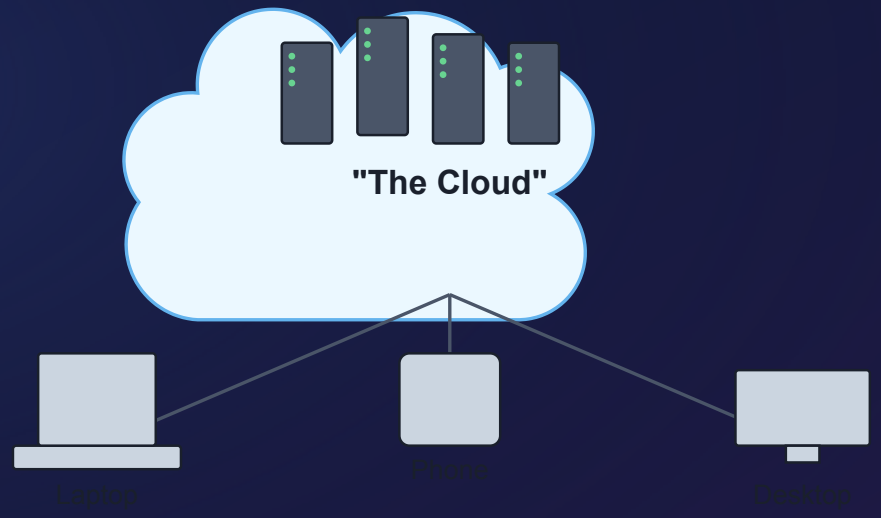
Bonus Floor — Computers Running Somewhere Else

Wait — does the app run on *my* computer?

- Often, only part of it does
- When you check the weather, chat, or stream a video, your device sends a request over the internet
- That request is handled by **another computer**, somewhere else entirely

So what *is* "the cloud"?

- Not a mystical thing floating above us
- It's just **someone else's computers** — running in a building far away
- Same electrons, same transistors, same CPU, same OS, same apps we just learned about
- The only difference: it's not sitting on your desk



Inside a data center

- Warehouses full of **racks** of computers
- Each rack holds many servers, each server has CPUs, memory, storage — exactly like the machine on your desk, just bigger and more of them
- Owned by companies (Amazon, Google, Microsoft, etc.) who rent out computing power to others



Why use someone else's computer?

- **Scale** — instantly use 1 computer or 10,000, only when you need them
- **Maintenance** — someone else keeps the hardware running, updated, and cooled
- **Access anywhere** — your data/app is reachable from any device, anywhere
- **Cost** — pay only for what you use, instead of buying your own hardware

Same tower, just far away 🔁

- Your phone/laptop is still climbing its own tower: apps → OS → CPU → bits → electrons
- It just also **talks to another tower**, in a data center, over a network
- "The cloud" = a lot of familiar towers, networked together, out of sight

🔗 Bridge: Bonus Floor → Recap

From nine floors back to one idea

- **We just climbed:** electrons → transistors → gates → CPU → bits → machine code → C → higher-level languages → OS → apps → the cloud
- **Next:** let's walk back down the *whole* tower in one breath, floor by floor, so it clicks as a single connected idea rather than nine separate ones

Putting It All Together

The full ladder, one line each

1. **Electrons** flow through transistors (switches)
2. Transistors form **logic gates**
3. Logic gates form a **CPU** that runs **bits & bytes**
4. The CPU executes **machine code**, written via **assembler**
5. **C** gives structure while staying close to the machine
6. **Higher-level languages** let us think more like humans
7. The **operating system** shares hardware between programs
8. **Applications** are what we actually see and use
9. **The cloud** is just more of these towers, somewhere else

Programming is choosing which floor of this tower to work on.

Questions?

Image Credits

- Diagrams (tower, cloud, hero title, hero background): original, made for this deck
- Logic gate symbols, voltage/binary signal diagram, binary background image, ASCII table: provided by the presenter
- Transistors, silicon wafer, CPU die, punched cards: ArnoldReinhold / Sangitiana Fararano / Revaldinho — CC BY / CC BY-SA, Wikimedia Commons
- Keyboard, smartphone, data center: Jovonni Pharr / Gannu03 / Carl Lender — CC BY / CC BY-SA, Wikimedia Commons
- Tux: Larry Ewing / Simon Budig / Anja Gerwinski, Wikimedia Commons

(full details in `images/CREDITS.md`)